

Dependency Inversion Principle in C# with Examples

In this article, I will discuss the **Dependency Inversion Principle in C#** with Examples. Please read our previous article before proceeding to this article, where we discussed the [Interface Segregation Principle in C#](#) with Examples. The Letter **D** in SOLID stands for the **Dependency Inversion Principle**, also known as **DIP**. You will understand the following pointers in detail at the end of this article.

1. [What is the Dependency Inversion Principle in C#?](#)
2. [Example without using Dependency Inversion Principle in C#](#)
3. [Example using Dependency Inversion Principle in C#](#)
4. [Advantages and Disadvantages of using the Dependency Inversion Principle in C#](#)
5. [How Do We Use the Dependency Inversion Principle in C#?](#)
6. [Advantages of Dependency Inversion Principle in C#.](#)

What is the Dependency Inversion Principle in C#?

The **Dependency Inversion Principle (DIP)** states that **High-Level Modules/Classes should not depend on Low-Level Modules/Classes. Both should depend upon Abstractions (e.g., interfaces or abstract classes). Secondly, Abstractions should not depend upon Details. But Details should depend upon Abstractions.**

The most important point you need to remember while developing real-time applications is always to keep the High-level and Low-level modules as loosely coupled as possible.

When a class knows about the design and implementation of another class, it raises the risk that if we make any changes to one class (e.g., modifying the method signature or return type or even renaming the class name), it will break the other class (the class consuming the other class members by creating an instance). So, we must keep these high-level and low-level modules/classes loosely coupled as much as possible. To do that, we need to make both of them dependent on abstractions instead of knowing each other.

Example to Understand Dependency Inversion Principle in C#

Let us understand the Dependency Inversion Principle with one Example using C# Language. First, we will see the example without following the Dependency Inversion Principle. Then, we will identify the problems of not following the Dependency Inversion Principle. Then, we will rewrite the same example using the Dependency Inversion Principle so that you will understand this concept easily. First, create a Console Application and then add the following class files.

Employee.cs

Create a class file named **Employee.cs** and copy and paste the following code. The following is a simple class having 4 properties. The following class is going to hold the employee data.

```
namespace SOLID_PRINCIPLES.DIP
{
    public class Employee
    {
        public int ID { get; set; }
    }
}
```

```

        public string Name { get; set; }
        public string Department { get; set; }
        public int Salary { get; set; }
    }
}

```

EmployeeDataAccessLogic.cs

Create a class file named **EmployeeDataAccessLogic.cs** and copy and paste the following code. The following class contains one method that takes the employee id and returns that Employee information. In a real-time application, you need to write the logic to get the employee details from the database, but for simplicity, we have hard-coded the employee details here. Also, in a real-time application, you might have more methods like getting all employee information, creating and updating employees, deleting employees, etc. Here, we have created only one method. So, this is the layer that will contain the logic to interact with the database, and hence, we said this class as EmployeeDataAccessLogic.

```

namespace SOLID_PRINCIPLES.DIP
{
    public class EmployeeDataAccessLogic
    {
        public Employee GetEmployeeDetails(int id)
        {
            //In real time get the employee details from database
            //but here we have hard coded the employee details
            Employee emp = new Employee()
            {
                ID = id,
                Name = "Pranaya",
                Department = "IT",
                Salary = 10000
            };
            return emp;
        }
    }
}

```

DataAccessFactory.cs

Create a class file named **DataAccessFactory.cs** and copy and paste the following code. The following class contains one static method, returning an instance of the EmployeeDataAccessLogic class. If you want to consume any method of the EmployeeDataAccessLogic class, then you need to create an instance of that class. In our example, the following class, the GetEmployeeDataAccessObj() static method, is going to return an instance of the EmployeeDataAccessLogic class, and using that instance, we can access the GetEmployeeDetails(int id) method. So, this class will return an instance of the EmployeeDataAccessLogic class, using which we can do the database operations.

```

namespace SOLID_PRINCIPLES.DIP
{
    public class DataAccessFactory
    {
        public static EmployeeDataAccessLogic GetEmployeeDataAccessObj()

```

```

    {
        return new EmployeeDataAccessLogic();
    }
}

```

EmployeeBusinessLogic.cs

Create a class file named **EmployeeBusinessLogic.cs** and copy and paste the following code. The following class has one constructor used to create an instance of the `EmployeeDataAccessLogic` class. Here, within the constructor, we call the static `GetEmployeeDataAccessObj()` method on the `DataAccessFactory` class, which will return an instance of `EmployeeDataAccessLogic`, and we initialize the `_EmployeeDataAccessLogic` property with the return instance. We also have one method, i.e., `GetEmployeeDetails`, which calls the `GetEmployeeDetails` method on the `EmployeeDataAccessLogic` instance to get the employee details by employee ID.

```

namespace SOLID_PRINCIPLES.DIP
{
    public class EmployeeBusinessLogic
    {
        EmployeeDataAccessLogic _EmployeeDataAccessLogic;
        public EmployeeBusinessLogic()
        {
            _EmployeeDataAccessLogic = DataAccessFactory.GetEmployeeDataAccessObj();
        }
        public Employee GetEmployeeDetails(int id)
        {
            return _EmployeeDataAccessLogic.GetEmployeeDetails(id);
        }
    }
}

```

Comparing the above Example with the Dependency Inversion Principle in C#

According to the Dependency Inversion Principle definition, “a **High-Level module should not depend on Low-Level modules. Both should depend on the abstraction.**”

So, first, we need to figure out the High-Level Module (class) and the Low-Level Module (class) in our example. A High-Level Module is a module that always depends on other modules. So, in our example, the `EmployeeBusinessLogic` class depends on the `EmployeeDataAccessLogic` class, so here, the `EmployeeBusinessLogic` class is the high-level module, and the `EmployeeDataAccessLogic` class is the low-level module.

So, as per the first rule of the Dependency Inversion Principle in C#, the `EmployeeBusinessLogic` class/module should not depend on the concrete `EmployeeDataAccessLogic` class/module. Instead, both classes should depend on abstraction. But, in our example, the way we have implemented the code, the `EmployeeBusinessLogic`, depending on the `EmployeeDataAccessLogic` class, means the first rule we are not following. In the later part of this article, I will modify the example to follow the Dependency Inversion Principle.

The second rule of the **Dependency Inversion Principle** states that “**Abstractions should not depend on details. Details should depend on Abstractions**”. Before understanding this, let us first understand what is an abstraction.

What is Abstraction in C#?

In simple words, we can say that Abstraction means Non-Concrete. So, Abstraction in Programming means we must create either an Interface or an Abstract Class, which is Non-Concrete, so we can not create an instance of it. In our example, the `EmployeeBusinessLogic` and `EmployeeDataAccessLogic` are concrete classes, meaning we can create objects of them. That means we are also not following the second rule of the Dependency Inversion Principle.

As per the Dependency Inversion Principle in C#, the `EmployeeBusinessLogic` (High-Level Module) should not depend on the concrete `EmployeeDataAccessLogic` (Low-Level Module) class. Both classes should depend on Abstractions, meaning both classes should depend on an Interface or an Abstract Class.

What should be in Interface (or in Abstract Class)?

As you can see in the above example, `EmployeeBusinessLogic` uses the `GetEmployeeDetails()` method of the `EmployeeDataAccessLogic` class. In real-time, there will be many employee-related methods in the `EmployeeDataAccessLogic` class. So, we need to declare the `GetEmployeeDetails(int id)` method or any employee-related methods within the interface or abstract class. By default, interface methods are going to be abstract. But if you create an abstract class, you must explicitly declare the methods as abstract by using the `abstract` keyword. I am going with Interface as it makes the code more loosely coupled, and we can achieve multiple inheritances.

IEmployeeDataAccessLogic.cs

Create a class file named **IEmployeeDataAccessLogic.cs**, then copy and paste the following code. As you can see, we created the interface with one abstract method, i.e., `GetEmployeeDetails`. You must declare those methods here if you have multiple employee-related methods.

```
namespace SOLID_PRINCIPLES.DIP
{
    public interface IEmployeeDataAccessLogic
    {
        Employee GetEmployeeDetails(int id);
        //Any Other Employee Related Method Declarations
    }
}
```

Next, we need to implement the **IEmployeeDataAccessLogic** in the **EmployeeDataAccessLogic** class. So, modify the **EmployeeDataAccessLogic** class as shown below. Here, you can see the `EmployeeDataAccessLogic` class implementing the `IEmployeeDataAccessLogic` class and providing implementations for the `GetEmployeeDetails` method.

```
namespace SOLID_PRINCIPLES.DIP
{
    public class EmployeeDataAccessLogic : IEmployeeDataAccessLogic
    {
```

```

    public Employee GetEmployeeDetails(int id)
    {
        //In real time get the employee details from database
        //but here we have hard coded the employee details
        Employee emp = new Employee()
        {
            ID = id,
            Name = "Pranaya",
            Department = "IT",
            Salary = 10000
        };
        return emp;
    }
}

```

Next, we need to change the DataAccessFactory class. Here, we need to change the return type of the GetEmployeeDataAccessObj to IEmployeeDataAccessLogic instead of EmployeeDataAccessLogic. Internally, the method creates an instance of the EmployeeDataAccessLogic class, but we return that instance to the user using the Parent Interface, i.e., IEmployeeDataAccessLogic. This is possible because a Parent Class Reference Variable can hold the child class object reference. Here, IEmployeeDataAccessLogic is the Parent class, and EmployeeDataAccessLogic is the Child class of the IEmployeeDataAccessLogic Parent class.

```

namespace SOLID_PRINCIPLES.DIP
{
    public class DataAccessFactory
    {
        public static IEmployeeDataAccessLogic GetEmployeeDataAccessObj()
        {
            return new EmployeeDataAccessLogic();
        }
    }
}

```

We need to change the EmployeeBusinessLogic class to use the IEmployeeDataAccessLogic instead of the concrete EmployeeDataAccessLogic class, as shown below. The EmployeeBusinessLogic class does not use the concrete EmployeeDataAccessLogic class; instead, it uses the non-concrete IEmployeeDataAccessLogic class.

```

namespace SOLID_PRINCIPLES.DIP
{
    public class EmployeeBusinessLogic
    {
        IEmployeeDataAccessLogic _IEmployeeDataAccessLogic;
        public EmployeeBusinessLogic()
        {
            _IEmployeeDataAccessLogic = DataAccessFactory.GetEmployeeDataAccessObj();
        }
        public Employee GetEmployeeDetails(int id)
        {

```

```

        return _IEmployeeDataAccessLogic.GetEmployeeDetails(id);
    }
}

```

That's it. We have implemented the Dependency Inversion Principle in our example using C# language where the High-Level module (EmployeeBusinessLogic) and Low-Level module (EmployeeDataAccessLogic) depend on abstraction (IEmployeeDataAccessLogic). Also, abstraction (IEmployeeDataAccessLogic) does not depend on details (EmployeeDataAccessLogic), but details depend on abstraction.

Now, you can test whether the application code is working as expected or not by modifying the Main method of the Program class as follows. Here, we are simply creating an instance of the EmployeeBusinessLogic class, calling the GetEmployeeDetails method, and then printing the employee details on the Console window.

```

using System;
namespace SOLID_PRINCIPLES.DIP
{
    public class Program
    {
        static void Main(string[] args)
        {
            EmployeeBusinessLogic employeeBusinessLogic = new EmployeeBusinessLogic();
            Employee emp = employeeBusinessLogic.GetEmployeeDetails(1001);
            Console.WriteLine($"ID: {emp.ID}, Name: {emp.Name}, Department: {emp.Department}, Salary: {emp.Salary}");
            Console.ReadKey();
        }
    }
}

```

When you run the above code, you will get the following output.

```
ID: 1001, Name: Pranaya, Department: IT, Salary: 10000
```

The EmployeeBusinessLogic and EmployeeDataAccessLogic classes are loosely coupled because EmployeeBusinessLogic does not depend on the concrete EmployeeDataAccessLogic class. Instead, it includes a reference to the IEmployeeDataAccessLogic interface. So now, we can easily use another class that implements IEmployeeDataAccessLogic with a different implementation, and for that, we don't need to make any changes to the EmployeeBusinessLogic class.

Advantages of Dependency Inversion Principle in C#

The following are the key advantages of applying the Dependency Inversion Principle in C#:

- **Reduced Dependencies Between Modules:** By depending on abstractions rather than concrete implementations, modules become less tightly coupled. This reduction in dependency makes the system easier to modify, extend, or refactor without affecting other parts of the system.
- **Easier Maintenance and Updating:** Since high-level modules are not tightly coupled to low-level modules, updates or changes to the implementation details of a system do not necessitate changes in

the high-level modules. This separation simplifies maintenance and the introduction of new features.

- **Improved Testability:** Dependency inversion facilitates the use of mocking frameworks, allowing developers to write unit tests that are independent of external resources or complex dependencies. Testing high-level logic without worrying about the details of its dependencies leads to more robust and reliable tests.
- **Facilitates Parallel Development:** Since components are decoupled and interact through abstractions, different parts of the system can be developed in parallel. Teams can work on separate components simultaneously without waiting for others to complete their tasks, which speeds up the development process.

In the next article, I will discuss [Multiple Real-Time Examples of the Dependency Inversion Principle \(DIP\) in C#](#). In this article, I explain the **Dependency Inversion Principle in C#** with Examples. I hope this article will help you with your needs. I would like to have your feedback. Please post your feedback, questions, or comments about this Dependency Inversion Principle article. With this, we have completed SOLID Design Principles using C# Language. Please give your valuable feedback in the comment section.